

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA5116 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO5: *Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.CO (BL5, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A

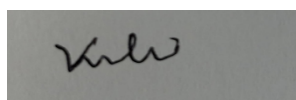
CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct:

I _____VARSHA.S/192511358_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

CONSTRAINT-BASED PROBLEM SOLVING

1. Intrusion Detection Deployment under Real-Time Constraints

A financial organization plans to deploy an Intrusion Detection System (IDS) with the following constraints:

- Must analyze traffic in real-time with latency ≤ 5 ms per packet
- Detection accuracy $\geq 96\%$
- False positive rate $\leq 1.5\%$
- Network traffic load: 150,000 packets per second
- Limited CPU resources (shared with other services)

Task:

Design an optimized IDS configuration (signature-based, anomaly-based, or hybrid). Analyze trade-offs between detection accuracy and system performance, and justify how your solution meets all constraints.

2. Worm Containment Strategy under Network Availability Constraints

An enterprise network faces rapid worm propagation with the following constraints:

- Network downtime must be $\leq 1\%$
- Detection and containment must occur within 2 minutes
- Systems are distributed across multiple locations
- Limited centralized control

Task:

Design a containment strategy using firewalls, IDS/IPS, and segmentation. Evaluate trade-offs between rapid isolation and business continuity, and justify how your approach minimizes spread while maintaining network availability.

3. Firewall Rule Optimization under Performance Constraints

A company firewall currently uses 10,000+ rules, causing performance degradation. Constraints include:

- Throughput requirement: ≥ 800 Mbps
- Rule processing time must be minimized
- Must maintain strict access control policies
- No additional hardware allowed

Task:

Analyze inefficiencies in rule processing. Propose optimization techniques (rule aggregation, prioritization, redundancy elimination). Justify how your optimized rule set improves performance while maintaining security.

4. IPS Deployment under Resource and Accuracy Constraints

An organization wants to deploy an Intrusion Prevention System (IPS) with the following constraints:

- Must block $\geq 90\%$ of known attacks
- False positive rate $\leq 3\%$
- Available memory for signatures: 1 GB
- Network speed: 500 Mbps

Task:

Design an IPS configuration strategy (signature selection, traffic filtering, adaptive updates). Evaluate trade-offs between resource usage and detection capability, and justify how your solution satisfies all constraints.

5. Malware Detection using Behavioral Analysis under System Constraints

A company plans to implement behavioral-based malware detection with the following constraints:

- Endpoint CPU usage must remain $\leq 15\%$
- Detection must occur within 10 seconds of execution
- Must detect unknown (zero-day) threats
- Limited storage for logs and analysis

Task:

Design a lightweight behavioral detection system. Analyze trade-offs between detection speed, accuracy, and resource consumption, and justify how your approach effectively identifies malware within the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and	Provides reasonable analysis	Limited analysis; weak	No meaningful analysis or
		prioritizes them logically	with some prioritization	prioritization	prioritization

Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Intrusion Detection Deployment under Real-Time Constraints

Problem Understanding

A financial organization plans to deploy an Intrusion Detection System (IDS) to monitor network traffic in real time. The IDS must satisfy several constraints:

- Analyze packets with a latency of **5 ms or less**.
- Achieve **96% or higher detection accuracy**.
- Maintain a **false positive rate below 1.5%**.
- Handle **150,000 packets per second**.
- Operate efficiently with **limited CPU resources** because the CPU is shared with other applications.

The challenge is to design an IDS that offers excellent security without reducing network performance.

Optimized IDS Configuration

A **Hybrid IDS**, which combines **Signature-based Detection** and **Anomaly-based Detection**, is the most appropriate solution.

Signature-Based Detection

Signature-based IDS compares incoming packets against a database of known attack signatures. It is highly effective for detecting malware, viruses, worms, Trojans, SQL injection, and denial-of-service attacks that have already been identified.

Advantages

- Very fast packet inspection.
- Low false positive rate.
- Minimal CPU utilization.
- Excellent for detecting known threats.

Limitations

- Cannot detect zero-day attacks.
- Requires frequent signature updates.

Anomaly-Based Detection

Anomaly-based IDS establishes a normal network behavior profile using machine learning or statistical analysis. Any significant deviation from the normal pattern is flagged as suspicious.

Advantages

- Detects unknown and zero-day attacks.
- Identifies insider threats.
- Detects new malware variants.

Limitations

- Higher computational requirements.
- Slightly higher false positive rate if not properly trained.

Optimized Design

The IDS should perform signature matching on every packet because this process is extremely fast. Only packets that appear suspicious should be forwarded to the anomaly detection engine. This layered approach minimizes CPU usage while maintaining high detection accuracy.

Additional optimization techniques include:

- Multi-threaded packet processing
- Parallel processing on multiple CPU cores
- Traffic prioritization
- Efficient packet buffering
- Incremental signature updates
- Load balancing among IDS sensors

Constraint Analysis

Constraint	Solution
Latency ≤ 5 ms	Fast signature matching and parallel processing
Accuracy $\geq 96\%$	Hybrid detection approach
False positives $\leq 1.5\%$	Threshold tuning and continuous learning
150,000 packets/sec	Distributed processing and load balancing
Limited CPU	Selective anomaly analysis

Trade-offs

A signature-based IDS provides high speed but cannot detect unknown attacks. An anomaly-based IDS detects new attacks but consumes more processing power. Combining both methods balances detection capability and system performance.

Justification

The hybrid IDS provides fast detection of known threats while maintaining the ability to detect zero-day attacks. The use of parallel processing and selective anomaly analysis ensures that latency remains below 5 milliseconds, detection accuracy exceeds 96%, false positives remain below 1.5%, and CPU usage stays within acceptable limits.

2. Worm Containment Strategy under Network Availability Constraints

Problem Understanding

An enterprise network experiences rapid worm propagation. The solution must:

- Detect and contain worms within **2 minutes**.
- Maintain **network downtime below 1%**.
- Protect systems distributed across multiple locations.
- Function with limited centralized management.

Containment Strategy

A layered security approach combining **firewalls**, **IDS**, **IPS**, and **network segmentation** is recommended.

Firewall Configuration

Firewalls should immediately block:

- Unauthorized ports
- Suspicious IP addresses
- Worm communication protocols
- Unnecessary inbound traffic

Firewalls should also implement Access Control Lists (ACLs) to restrict unauthorized communication.

Intrusion Detection System

The IDS continuously monitors traffic for worm-like behavior such as:

- Rapid scanning
- Multiple connection attempts
- Excessive bandwidth usage
- Repeated login failures

Upon detection, alerts are immediately sent to administrators.

Intrusion Prevention System

The IPS automatically:

- Blocks malicious packets
- Disconnects infected hosts
- Prevents further worm communication
- Updates firewall rules dynamically

Network Segmentation

The network is divided into VLANs based on departments. Even if one VLAN becomes infected, the worm cannot spread easily to other segments.

Endpoint Isolation

Compromised systems are quarantined automatically while unaffected systems continue operating normally.

Constraint Analysis

Constraint	Solution
Downtime $\leq 1\%$	Isolate only infected devices
Detection within 2 min	Real-time IDS monitoring
Distributed systems	Local IDS and IPS deployment
Limited centralized control	Autonomous local security devices

Trade-offs

Rapid isolation minimizes worm spread but may temporarily disconnect affected users. Allowing limited communication improves business continuity but slightly increases infection risk.

Justification

Using firewalls, IDS, IPS, segmentation, and endpoint isolation together creates multiple layers of defense. This approach quickly contains worm outbreaks while ensuring that business operations continue with minimal downtime.

3. Firewall Rule Optimization under Performance Constraints

Problem Understanding

The organization's firewall contains more than **10,000 rules**, leading to slow packet processing and reduced throughput. The firewall must maintain:

- Throughput of **800 Mbps or more**
- Fast rule processing
- Strong access control
- No additional hardware

Causes of Performance Degradation

- Duplicate rules
- Shadowed rules
- Unused rules
- Poor rule ordering
- Large rule database

Optimization Techniques

Rule Aggregation

Combine similar IP ranges and services into single rules.

Rule Prioritization

Place frequently matched rules near the top of the firewall rule list.

Redundancy Elimination

Remove:

- Duplicate rules
- Expired rules
- Unused entries

Policy Simplification

Use:

- Network object groups
- Service groups
- Wildcard addressing

Regular Rule Audits

Review rules periodically to eliminate obsolete policies.

Constraint Analysis

Constraint	Solution
Throughput $\geq$ 800 Mbps	Reduce rule comparisons

Faster processing	Rule prioritization
Strict security	Maintain access control policies
No new hardware	Software optimization only

Trade-offs

Aggressive rule reduction improves performance but must be carefully tested to avoid accidentally removing important security policies.

Justification

Optimizing firewall rules reduces lookup time, improves throughput, decreases CPU utilization, and preserves strong security without requiring hardware upgrades.

4. IPS Deployment under Resource and Accuracy Constraints

Problem Understanding

The organization requires an Intrusion Prevention System that:

- Blocks **90% or more** of known attacks
- Maintains **false positives below 3%**
- Uses only **1 GB memory**
- Supports **500 Mbps** network traffic

IPS Configuration Strategy

Signature Selection

Store only frequently occurring attack signatures.

Remove:

- Obsolete signatures
- Duplicate signatures
- Rare attack signatures

Traffic Filtering

Inspect only:

- External traffic
- High-risk ports
- Critical applications

Trusted internal traffic receives lighter inspection.

Adaptive Updates

Automatically download:

- Critical signatures immediately
- Full updates during non-peak hours

Memory Optimization

Compress the signature database and remove inactive entries.

Constraint Analysis

Constraint	Solution
------------	----------

≥90% attack blocking	Optimized signature database
≤3% false positives	Fine-tuned detection rules
1 GB memory	Signature compression
500 Mbps	Selective inspection

Trade-offs

A smaller signature database reduces memory usage and improves speed but may slightly reduce coverage of rare attacks. Frequent updates compensate for this limitation.

Justification

Prioritized signatures, adaptive updates, and selective traffic inspection allow the IPS to meet all performance and detection requirements while operating within memory constraints.

5. Malware Detection using Behavioral Analysis

Problem Understanding

The organization wants a behavioral malware detection system that:

- Uses **15% or less CPU**
- Detects malware within **10 seconds**
- Identifies unknown (zero-day) malware
- Uses limited storage

Behavioral Detection System

Instead of searching for virus signatures, the system continuously monitors application behavior.

Behavior Monitoring

The system observes:

- Process creation
- Registry modifications
- File encryption
- Memory allocation
- Network communication
- DLL injection
- Privilege escalation
- Unauthorized file access

Detection Engine

A lightweight rule engine identifies suspicious activities such as:

- Rapid file encryption
- Multiple process spawning
- Unexpected registry changes
- Connections to malicious IP addresses

Machine learning models further classify suspicious behavior.

Log Management

To reduce storage:

- Store only suspicious events.
- Compress log files.
- Automatically delete outdated logs.
- Send critical events to a central server.

Constraint Analysis

Constraint	Solution
CPU $\leq$ 15%	Lightweight monitoring
Detection $\leq$ 10 sec	Real-time behavior analysis
Zero-day detection	Machine learning and behavior monitoring
Limited storage	Compressed event logging

Trade-offs

Behavior-based detection consumes slightly more CPU than signature-based methods but provides much better protection against unknown threats. Careful tuning reduces false alarms while maintaining rapid detection.

Justification

A lightweight behavioral detection system provides fast identification of both known and unknown malware. By monitoring runtime activities rather than relying solely on signatures, the system effectively detects zero-day attacks while remaining within CPU and storage limits. This satisfies all the given constraints and provides strong endpoint protection.